

Giorgio De Santis

Robotics & AI Engineer | Percezione, Controllo e Computer Vision

desantisgiorgio20@gmail.com | github.com/giorgio0420 | Roma, Italia

Profilo Professionale

Studente di Laurea Magistrale in Artificial Intelligence and Robotics presso Sapienza Università di Roma (laurea prevista: marzo 2027), con alle spalle una Laurea Triennale in Ingegneria Elettronica. Esperienza pratica sull'intera pipeline robotica-AI: derivazione e calibrazione di controllori a impedenza/forza per manipolatori industriali, costruzione di stack di navigazione ROS 2 e interazione sociale, addestramento di modelli di visione basati su transformer su immagini satellitari, il tutto su una solida base elettronica (microcontrollori, DSP, progettazione circuitale). Al di fuori dei corsi, progettazione e messa in produzione di piccoli sistemi software end-to-end – tra cui una pipeline di contenuti completamente automatizzata su base giornaliera – e, per pura curiosità, progetti personali di fisica computazionale (modelli di Ising, classificazione di eventi LHC).

Istruzione

Laurea Magistrale, Artificial Intelligence and Robotics — Sapienza Università di Roma

Laurea prevista: marzo 2027

Corsi rilevanti: Neural Networks, Robotics 1 & 2, Control Theory, Computer Vision, Model Predictive Control

Laurea Triennale, Ingegneria Elettronica — Sapienza Università di Roma (2024)

Tesi: “Machine Learning Applications in Electromagnetics” | Focus: architetture a microcontrollore, elaborazione digitale dei segnali, C/C++ embedded, teoria dei circuiti, controlli automatici – la base hardware/segnali che oggi sostiene il mio approccio alla robotica embedded e al controllo (vedi Progetti di Robotica e Controllo)

Competenze Tecniche

Linguaggi: Python, C++, C, MATLAB, Lua

Robotica: ROS 2, Gazebo, RViz, CoppeliaSim, controllo a impedenza/forza, proiezione nel null space, obstacle avoidance, (LPV-)MPC

AI / ML: PyTorch, TensorFlow/Keras, scikit-learn, OpenCV, Hugging Face, TorchGeo, Swin Transformer, U-Net, CNN, LSTM, random forest

Embedded & Elettronica: STM32, Arduino, LTspice (+ TINA-TI)

Piattaforme & Strumenti: Git, Linux, Docker, LaTeX

Progetti di Robotica e Controllo

Controllo a Impedenza Dinamico per Sonda Ecografica Teleoperata — Python, CoppeliaSim

- Progettato e calibrato un controllore a impedenza in Python per un braccio KUKA LBR iiwa a 7 gradi di libertà, sostituendo il controllo di posizione rigido con una teleoperazione compliant e a forza limitata.
- Costruito e validato l'intero loop di teleoperazione e controllo direttamente in CoppeliaSim, ottenendo un contatto sonda-superficie stabile e sicuro per l'intera durata di traiettorie di scansione ecografica simulate.

Obstacle Avoidance nel Null Space per Manipolatori Ridondanti — MATLAB, CoppeliaSim, KUKA LBR iiwa 7

- Sfruttata la ridondanza cinematica del manipolatore KUKA LBR iiwa a 7 gradi di libertà per proiettare il moto di obstacle avoidance nel null space mantenendo il tracking di una traiettoria cartesiana fissa dell'end-effector, disaccoppiando il task principale dal comportamento di evitamento.

LPV-MPC per il Controllo di un Hexarotor Sottoattuato — ROS 2, Gazebo

- Implementazione di uno schema Linear-Parameter-Varying Model Predictive Control (LPV-MPC) in ROS 2/Gazebo per disaccoppiare e stabilizzare le dinamiche traslazionali e rotazionali (Zero-Moment-Point) di un drone hexarotor sottoattuato, come capstone del corso di Aerial Robotics.

Obstacle Avoidance per UAV tramite Campi Vettoriali a Vortice — Python, CoppeliaSim

- Contribuito a uno schema di navigazione basato su campi vettoriali a vortice per l'obstacle avoidance di UAV, generando traiettorie fluide e prive di collisioni attorno agli ostacoli in ambienti simulati e affollati.

CareAssist-HRI: Robot Sociale Cognitivo (fase iniziale) — ROS 2

- In sviluppo: costmap sociali e logica di interazione perché un robot socialmente consapevole navighi e interagisca in modo appropriato tra le persone, non solo attorno agli ostacoli statici – progetto ancora in corso, non concluso.

Progetti di AI e Computer Vision

Segmentazione di Immagini Satellitari con Swin Transformer e U-Net — PyTorch, TorchGeo

- Implementate e confrontate architetture di segmentazione U-Net e Swin Transformer per la segmentazione semantica del suolo ad alta risoluzione.
- Inizializzati i backbone con pesi pre-addestrati SatMAE++ e misurato l'effetto del pre-training specifico per il telerilevamento rispetto all'addestramento da zero, come progetto del corso di Neural Networks.

CNN vs. LSTM: l'Architettura Segue la Forma dei Dati — PyTorch

- Costruita una CNN per leggere la texture spaziale di configurazioni di spin del modello di Ising in 2D e una LSTM per leggere l'ordine temporale di sei serie di sensori di qualità dell'aria, usando lo stesso framework di regressione per mostrare come la scelta dell'architettura debba seguire la forma dei dati e non l'ambito da cui provengono.

Feature Fisiche vs. CNN su Immagini Grezze per Eventi LHC — scikit-learn, PyTorch

- Confrontato un classificatore random forest addestrato su feature fisiche costruite a mano con una CNN addestrata su immagini grezze 32x32 del rivelatore, sullo stesso insieme di eventi di collisione di fisica delle alte energie, per quantificare cosa impara in più il modello sull'immagine grezza rispetto alle feature esperte.

Immagini Dentali: Generazione Sintetica e Segmentazione — PyTorch, computer vision

- Contribuito a una pipeline di deep learning per la generazione sintetica e la segmentazione semantica di immagini dentali, con l'obiettivo di ampliare dataset medicali con poche etichette disponibili.

Progetti di Software Engineering e Personali

Cultura – Pipeline Automatizzata di Contenuti — Python, GitHub Actions, PWA

- Progettato e costruito end-to-end un sistema personale che filtra e organizza contenuti ad alto valore informativo su base giornaliera.
- Sviluppata una pipeline di trascrizione audio (yt-dlp + Groq Whisper API) eseguita su una VM cloud esterna, che alimenta un motore di digest e generazione di moduli di studio.
- Automatizzata l'intera pipeline con job schedulati su GitHub Actions e distribuita una PWA installabile come front-end di lettura – un percorso completo dall'ingestione dei dati a un prodotto distribuito e rivolto all'utente, progettato e mantenuto da solo.

Computational Physics Lab — C

- Implementati l'attrattore di Rössler, random walk, percolazione e un modello a gas reticolare in puro C per un corso di laboratorio universitario, con un target `make check` che ricalcola i risultati di riferimento e fa fallire la build in caso di deriva numerica – un'abitudine precoce a trattare il codice scientifico con lo stesso rigore del codice di produzione.